

Tutorial on Sorting Techniques

For Data Structure Students

Topics covered: stability and in-place properties, internal vs external sorting, Bubble/Selection/Insertion sort, lower bound for comparison-based sorting, recursive merge sort, and recursive quicksort.

Learning outcomes

- Differentiate between stable and unstable sorting algorithms.
- Identify which sorting methods are in-place and which need extra memory.
- Explain internal sorting and external sorting in simple terms.
- Trace Bubble, Selection, and Insertion sort step by step.
- Understand why comparison-based sorting cannot do better than $O(n \log n)$ in the general case.
- Describe the recursive idea behind Merge Sort and Quick Sort.

1. Stability and In-Place Properties

When we study sorting algorithms, we do not only ask “Is the data sorted?” We also ask how the algorithm behaves while sorting. Two important properties are stability and in-place behavior.

Stable sorting

A sorting algorithm is stable if equal elements keep the same relative order after sorting.

Example: Suppose records are sorted by marks, and two students both have 85 marks. If Student A appeared before Student B in the input, a stable sort keeps A before B in the output.

In-place sorting

A sorting algorithm is called in-place if it uses only a small, constant amount of extra memory apart from the input array.

In simple words, it sorts mostly inside the original array instead of creating another large array.

Example showing stability

Input order	Record	Key used for sorting
1	(A, 85)	85
2	(B, 70)	70
3	(C, 85)	85

If these records are sorted by marks in ascending order, the stable result is: (B,70), (A,85), (C,85). A and C have equal keys, and their original relative order is preserved.

Common examples

Algorithm	Stable?	In-place?	Main idea
Bubble Sort	Yes	Yes	Repeatedly swaps adjacent out-of-order elements
Selection Sort	No (usual version)	Yes	Selects minimum and puts it in correct position
Insertion Sort	Yes	Yes	Inserts each element into the sorted left part
Merge Sort	Yes	No	Divides array and merges sorted halves
Quick Sort	No (usual version)	Mostly yes	Partitions around a pivot

2. Internal Sorting vs External Sorting

Sorting methods can also be classified based on where the data is stored during sorting.

- Internal sorting: the entire data fits in main memory (RAM), so sorting is done in memory.
- External sorting: the data is too large to fit in RAM, so files on disk are used during sorting.

Internal sorting is common in classroom algorithms such as Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort. External sorting is used for very large datasets, for example huge database files or log files.

Simple comparison

Feature	Internal sorting	External sorting
Where data stays	Main memory	Disk + memory
Used when	Data size is manageable	Data size is very large

3. Bubble Sort, Selection Sort, and Insertion Sort

These three algorithms are easy to understand and are often taught first. They are useful for learning the logic of sorting, although they are not the fastest for large inputs.

3.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element “bubbles up” to its correct position.

Example

Sort the array: 5, 1, 4, 2

Pass 1: 5 1 4 2

Compare 5 and 1 -> swap -> 1 5 4 2

Compare 5 and 4 -> swap -> 1 4 5 2

Compare 5 and 2 -> swap -> 1 4 2 5

Pass 2: 1 4 2 5

Compare 1 and 4 -> no swap

Compare 4 and 2 -> swap -> 1 2 4 5

Pass 3: 1 2 4 5 (already sorted)

BubbleSort(A, n)

```
for i = 0 to n-2
  for j = 0 to n-2-i
    if A[j] > A[j+1]
      swap A[j], A[j+1]
```

Time complexity: $O(n^2)$ in the average and worst case. Best case can be $O(n)$ if an optimized version stops early when no swaps occur.

3.2 Selection Sort

Find minimum and
place in the first

Selection Sort finds the smallest element from the unsorted part and places it at the beginning. It reduces the unsorted region one position at a time.

Sort the array: 29, 10, 14, 37, 13

Pass 1: smallest is 10 -> swap with 29 -> 10 29 14 37 13

Pass 2: smallest in remaining part is 13 -> swap with 29 -> 10 13 14 37 29

Pass 3: smallest in remaining part is 14 -> already correct

Pass 4: smallest in remaining part is 29 -> swap with 37 -> 10 13 14 29 37

Selection Sort

```
for i = 0 to n - 2
  minIndex = i

  for j = i + 1 to n - 1
    if A[j] < A[minIndex]
      minIndex = j

  if minIndex != i
    swap A[i] and A[minIndex]
```

Time complexity: $O(n^2)$ in best, average, and worst cases. It performs fewer swaps than Bubble Sort.

3.3 Insertion Sort

Find Correct position

Insertion Sort builds the final sorted array one element at a time. It takes each new element and inserts it into the correct position in the already sorted left part.

Sort the array: 8, 3, 5, 4, 6

Start: 8 | 3 5 4 6

Insert 3 into sorted part -> 3 8 | 5 4 6

Insert 5 -> 3 5 8 | 4 6

Insert 4 -> 3 4 5 8 | 6

Insert 6 -> 3 4 5 6 8

Time complexity: $O(n^2)$ in average and worst cases, but $O(n)$ in the best case when the array is already nearly sorted. It works very well for small or nearly sorted inputs.

Comparison of the three basic sorts

Algorithm	Best case	Average case	Worst case	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes

4. Lower Bound for Comparison-Based Sorting

Comparison-based sorting algorithms make decisions by comparing elements, for example checking whether $a[i] < a[j]$. Examples include Merge Sort, Quick Sort, Heap Sort, Bubble Sort, Selection Sort, and Insertion Sort.

A very important result in data structures is that any comparison-based sorting algorithm needs at least $O(n \log n)$ comparisons in the worst case.

Why?

Think of sorting as deciding among all possible orders of n distinct elements. There are $n!$ possible permutations. A comparison can be seen as a yes/no decision, so after k comparisons we can distinguish at most 2^k cases.

To correctly sort every possible input, the algorithm must be able to identify one of all $n!$ possible permutations. Therefore:

$$2^k \geq n!$$

Taking logarithm on both sides gives:

$$k \geq \log_2(n!)$$

$$\text{and } \log_2(n!) = \Theta(n \log n)$$

Hence the lower bound is $\Omega(n \log n)$. This means no comparison-based sorting algorithm can be faster than this for all inputs in the general case.

That is why Merge Sort and well-implemented Quick Sort are considered efficient: they get close to this theoretical limit.

5. Recursive Merge Sort

Merge Sort follows the divide-and-conquer technique. It recursively divides the array into two halves, sorts each half, and then merges the two sorted halves.

Main idea

- Divide the array into two halves.
- Recursively sort the left half.
- Recursively sort the right half.
- Merge the two sorted halves into one sorted array.

Simple pseudocode

```
MergeSort(A, low, high):  
    if low < high:  
        mid = (low + high) // 2  
        MergeSort(A, low, mid)  
        MergeSort(A, mid + 1, high)  
        Merge(A, low, mid, high)
```

Time complexity: $O(n \log n)$ in best, average, and worst cases. It is stable, but it requires extra space for merging.

```
Merge(A, low, mid, high)
```

```
Create two temporary arrays:
```

```
Left = A[low ... mid]
```

```
Right = A[mid+1 ... high]
```

```
i = 0
```

```
j = 0
```

```
k = low
```

```
while i < size of Left and j < size of Right
```

```
    if Left[i] <= Right[j]
```

```
        A[k] = Left[i]
```

```
        i = i + 1
```

```
    else
```

```
        A[k] = Right[j]
```

```
        j = j + 1
```

```
    k = k + 1
```

```
while i < size of Left
```

```
    A[k] = Left[i]
```

```
    i = i + 1
```

```
    k = k + 1
```

```
while j < size of Right
```

```
    A[k] = Right[j]
```

```
    j = j + 1
```

```
    k = k + 1
```

6. Recursive Quick Sort

Quick Sort also uses divide-and-conquer, but it works differently. It chooses a pivot, places smaller elements on the left and larger elements on the right, and then recursively sorts the two parts.

Main idea

- Choose a pivot element.
- Partition the array so that elements smaller than the pivot come before it and larger elements come after it.
- Recursively apply Quick Sort to the left and right parts.

Example

```
Array: [9, 4, 8, 3, 1, 2, 5]
Choose pivot = 5
After partition: [4, 3, 1, 2] 5 [9, 8]
Recursively sort left part -> [1, 2, 3, 4]
Recursively sort right part -> [8, 9]
Final result -> [1, 2, 3, 4, 5, 8, 9]
```

Simple pseudocode

```
QuickSort(A, low, high):
    if low < high:
        p = Partition(A, low, high)
        QuickSort(A, low, p - 1)
        QuickSort(A, p + 1, high)
```

Average time complexity: $O(n \log n)$. Worst case: $O(n^2)$, which can happen when the pivot choice is poor, such as always selecting the first element in an already sorted array. In practice, random pivot or median-like strategies make Quick Sort very fast.

Quick Sort is usually not stable, but it is attractive because it often sorts in-place and performs very well in practice.

7. Quick Revision Table

Algorithm	Stable	In-place	Best	Average	Worst
Bubble Sort	Yes	Yes	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	No	Yes	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	Yes	Yes	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	Yes	No	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	No	Mostly yes	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Exam-oriented summary

- Stable means equal keys keep their relative order.
- In-place means the algorithm uses only a small extra amount of memory.
- Internal sorting is done completely in RAM; external sorting uses disk because data is huge.
- Bubble, Selection, and Insertion Sort are easy to learn but slow for large n because they are generally $O(n^2)$.
- Comparison-based sorting has a theoretical lower bound of $\Omega(n \log n)$.
- Merge Sort guarantees $O(n \log n)$ but needs extra space.
- Quick Sort is usually very fast in practice, but its worst case is $O(n^2)$.